Dstrmaysam Healthcare Knowledge Agent

Low-Level Design Document For SDLC Delivery

A developer-oriented design pack covering architecture, APIs, module logic, data design, agent workflows, operational controls, and non-functional requirements.

Document type	Low-Level Design / Detailed Design
System baseline	Current repository implementation after LangGraph + RAGAS/Langfuse changes
Deployment target	AWS ECS Fargate, S3, OpenSearch Serverless, DynamoDB, Secrets Manager, CloudWatch
Generated	2026-06-19

This document describes the implementation-level design for the system: architecture, API contracts, logging and exception handling, database design, technical specifications, UI flow details, and non-functional requirements.

1. LLD Purpose And SDLC Context

The LLD explains how the system works at implementation level. It describes module behavior, class/module responsibilities, database definitions, interfaces, sequence flows, and operational concerns so developers and reviewers can move from requirements to implementation and deployment without relying on implicit assumptions.

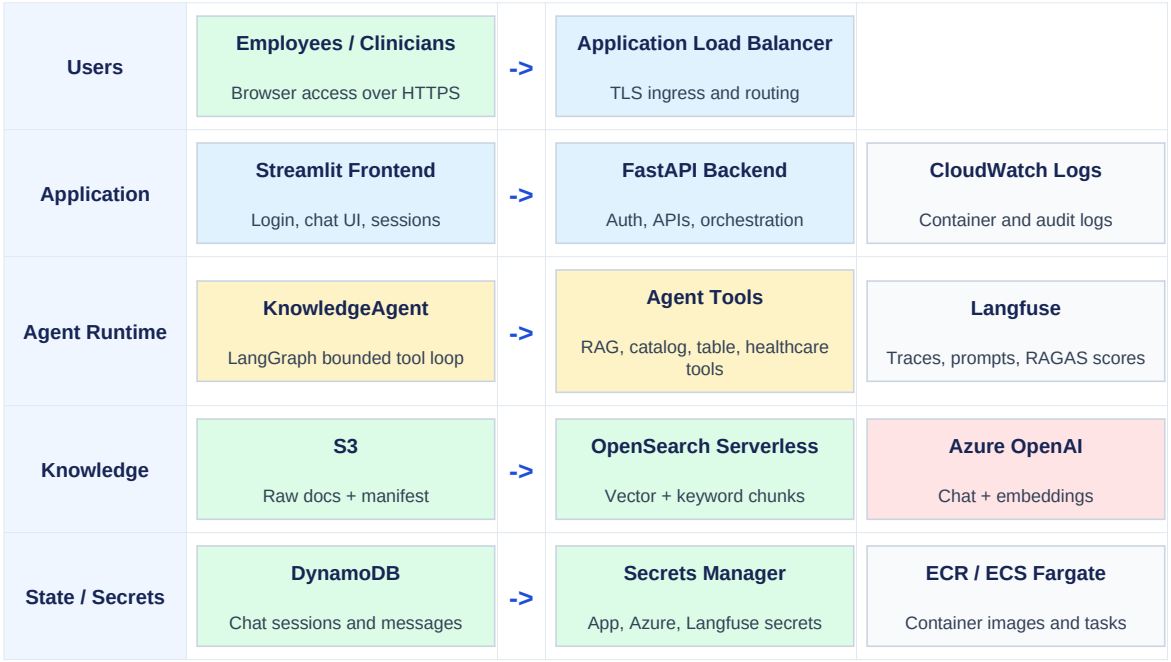
LLD Area	How This PDF Covers It
Architecture diagrams	Layered AWS and application architecture, plus workflow diagrams.
API details	FastAPI routes, request/response models, authentication, and expected behavior.
Logging and exceptions	CloudWatch logs, Langfuse traces, audit events, and fallback behavior.
Database design	DynamoDB key schema, OpenSearch mapping, S3 manifest, and Secrets Manager schemas.
Technical specifications	Module responsibilities, LangGraph loop, tools, ingestion, and eval logic.
UI detailing	Streamlit login/chat/session behavior and validation boundaries.
NFRs	Security, availability, performance, scalability, maintainability, and evaluation requirements.

2. System Scope

The Dstrmaysam Healthcare Knowledge Agent is a containerized internal assistant for grounded healthcare knowledge retrieval and answer generation. Users authenticate through a simple login UI, ask questions in Streamlit, and receive answers generated by a FastAPI backend that orchestrates LangGraph, Azure OpenAI, RAG over S3/OpenSearch, chat history, healthcare access controls, safety assessment, and Langfuse tracing.

Capability	Implementation Detail
Authenticated chat	Bearer tokens from `/auth/login`; token verified on chat/session/document routes.
Agent execution	KnowledgeAgent uses a LangGraph-wrapped bounded tool loop with LangChain-compatible Azure OpenAI model calls.
Grounding	Retrieval-backed tools capture citations and snippets from OpenSearch hits.
Healthcare controls	PHI redaction, role-based document filtering, safety flags, and audit event metadata.
Observability	Langfuse trace IDs are returned to `/chat`; RAGAS scores can publish back to Langfuse.
Persistence	DynamoDB stores chat sessions/messages in AWS dev; memory store supports local development.

3. Overall Architecture Diagram



The backend is the trusted boundary. It owns authentication, secret loading, model access, retrieval, source filtering, safety checks, history persistence, and observability. The frontend remains thin and does not connect directly to AWS knowledge stores, Azure OpenAI, Langfuse, or Secrets Manager.

4. Deployment Environment

Layer	Service	AWS Dev Value / Responsibility
Ingress	Application Load Balancer	HTTPS entry point; routes to Streamlit frontend.
Containers	ECS Fargate	Runs backend and frontend task definitions.
Images	ECR	`dstrmaysam-healthcare-knowledge-agent-backend`, `...-frontend`.
Documents	S3	`dstrmaysam-healthcare-knowledge-agent-dev`, raw docs and manifest.
Vector search	OpenSearch Serverless	Index `dstrmaysam-healthcare-knowledge-agent` with 1536-dim vectors.
History	DynamoDB	Table `dstrmaysam-healthcare-knowledge-agent-dev`.
Secrets	AWS Secrets Manager	`/dstrmaysam-healthcare-knowledge-agent/dev/{app,azure-openai,langfuse}`.
Logs	CloudWatch Logs	`/ecs/dstrmaysam-healthcare-knowledge-agent/backend` and `/frontend`.
LLM	Azure OpenAI	Chat and embedding deployments via `langchain-openai`.
Observability	Langfuse	Prompt, trace, span, and score publishing.

AWS dev ECS configuration sets `APP_ENV=dev`, `SECRETS_STAGE=dev`, `CHAT_HISTORY_BACKEND=dynamodb`, and `LOCAL_TEST_ADMIN_ENABLED=false`.

5. Application Module Design

Module	Responsibility	Key Files
API layer	FastAPI routes, dependency wiring, auth checks, response mapping.	`backend/app/main.py`, `models.py`
Auth service	PBKDF2 password verification, HMAC token creation/verification.	`backend/app/auth.py`
Agent runtime	PHI redaction, prompt assembly, LangGraph tool loop, safety/audit/history.	`backend/app/agent.py`
Tool layer	RAG, catalog, table lookup, healthcare-specific tools.	`tools.py`, `healthcare_tools.py`
Retrieval	OpenSearch query, Azure embedding query vector, hit normalization.	`retrieval.py`
Storage	S3 manifest loading, CSV text reads, structured row lookup.	`storage.py`
Ingestion	S3 scan, parsing, chunking, embeddings, OpenSearch indexing, manifest write.	`ingest.py`
Healthcare controls	Access control, PHI redaction, safety assessment, audit event.	`healthcare.py`
Observability	Langfuse trace, callback, prompt loading, fallback trace IDs.	`observability.py`
Evaluation	RAGAS runs, source snippets, Langfuse score publishing.	`evals/run_ragas_eval.py`

6. API Low-Level Design

Route	Auth	Input	Output / Behavior
/health GET	No	None	Status, public settings, registered tools.
/auth/login POST	No	`username`, `password`	Bearer token and expiry; 401 on invalid credentials.
/chat POST	Yes	`query`, optional `session_id`	Answer, sources, snippets, tools used, token counts, latency, trace ID, safety, audit event.
/chat/sessions GET	Yes	Bearer token	Current user's session summaries.
/chat/sessions/{id} GET	Yes	Session ID	Messages for the selected session.
/documents GET	Yes	Bearer token	Role-filtered document catalog.

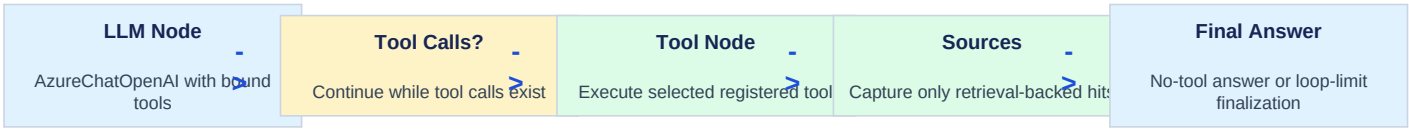
Model	Fields
ChatRequest	`query: str`, `session_id: str | None`
Source	`title`, `uri`, `score`, `metadata`, optional `snippet`
ChatResponse	`session_id`, `answer`, `sources`, `tools_used`, `input_tokens`, `output_tokens`, `latency_ms`, `trace_id`, `safety`, `audit_event`
ChatSessionSummary	`session_id`, `title`, `updated_at`
ChatSessionDetail	`session_id`, `messages[]`

7. Chat Workflow

Authenticated Chat Request
1. Streamlit sends `/chat` with bearer token, query, and optional session ID.
v
2. FastAPI verifies the token and builds `HealthcareUserContext` from claims.
v
3. KnowledgeAgent redacts PHI from prompt/trace inputs and loads bounded chat history.
v
4. Langfuse trace context is opened; system prompt and initial safety context are assembled.
v
5. LangGraph-backed tool loop lets the LLM choose tools and records only actual tool calls.
v
6. Retrieval-backed tools return sources with snippets; role filters remove inaccessible documents.
v
7. Final safety assessment and audit event are produced from the actual answer context.
v
8. User and assistant messages persist to DynamoDB in AWS dev; response returns trace ID and metadata.

Deterministic healthcare controls stay outside the model loop: PHI redaction happens before model input, initial safety context is included in the prompt, and the final safety assessment is run after sources and answer generation are known.

8. LangGraph Agent And Tool Workflow



Tool	Purpose	Source Capture
rag_search	Catalog-guided semantic RAG over indexed knowledge documents.	Yes
document_search	Catalog-guided semantic search over approved healthcare documents.	Yes
policy_search	Catalog-guided focused search over policies, SOPs, pathways, guidelines.	Yes
document_catalog	List/filter S3 manifest documents and provide internal candidate keys for RAG.	No
catalogue_search	Find services, owners, systems, departments, approved tools.	No
table_lookup	Exact lookup over CSV/table-like S3 files.	No
calendar_rota_lookup	Lookup rota, clinic, training, on-call schedule CSV rows.	No
formulary_table_lookup	Lookup formulary/restricted medicine/approval rows.	No
safety_guard	Detect clinical risk, missing sources, PHI exposure, escalation needs.	No

The graph is bounded to five LLM calls. If the limit is reached, the agent makes one final no-tool model call using accumulated tool results, preventing infinite tool-call loops.

Workflow	Low-Level Steps
RAG document search	Load manifest; match catalog terms against title/key/content type/metadata; keep up to 8 role-allowed keys; run OpenSearch vector or keyword query with optional OpenSearch key filter; return snippets and citations. If no candidate keys match, run broad retrieval.
Deterministic fact lookup	Load manifest; keep CSV/table sources; read matching S3 objects; parse with csv.DictReader; match query terms against row values; return exact JSON rows without semantic generation.
Document catalog	Load manifest; create DocumentRecord values; match terms against metadata; return document metadata for explicit tool calls, or candidate keys for internal RAG helper mode.

9. Ingestion And Retrieval Design

Document Ingestion	Retrieval
1. List S3 objects under `raw/`.	1. Load manifest and select up to 8 catalog candidate keys.
v	v
2. Parse PDF, DOCX, markdown, text, and CSV.	2. Embed user query with Azure OpenAI.
v	v
3. Infer healthcare metadata from object key.	3. Search OpenSearch `embedding` vector field with optional OpenSearch key filter.
v	v
4. Chunk text with overlap.	4. Fallback to keyword multi-match with optional OpenSearch key filter when embedding unavailab
v	v
5. Embed chunks with Azure OpenAI.	5. Normalize hits into title, URI, text, score, metadata.
v	v
6. Index chunks into OpenSearch.	6. Apply role-based access filtering.
v	v
7. Write S3 manifest to `manifests/documents.json`.	7. Return citations and snippets to the agent.

Store	Schema / Contract
S3 raw documents	`raw/` prefix contains PDF, DOCX, TXT, MD, CSV source files.
S3 manifest	`documents[]` with key, title, content_type, checksum, metadata, chunk_count.
OpenSearch index	`embedding` knn_vector 1536, `key`, `title`, `uri`, `text`, `content_type`, `chunk_index`, `checksum`, `metadata`.
DynamoDB history	Partition key `user_id`; sort key `sort_key`; session rows use `SESSION#...`; message rows use `MESSAGE#...`.
Secrets Manager	JSON documents for app auth, Azure OpenAI, and Langfuse credentials.

10. Database And Index Low-Level Design

DynamoDB Attribute	Role
user_id	Partition key. Every query is scoped to the authenticated user.
sort_key	Range key. Prefix distinguishes session summary rows and message rows.
session_id	Stable chat session identifier.
title	Derived from the first user message for session listing.
updated_at / created_at	ISO timestamps for ordering and display.
role / content / metadata	Message role, message text, and assistant response metadata.

OpenSearch Field	Type	Purpose
embedding	knn_vector dimension 1536	Semantic vector retrieval.
text	text	Chunk body used as answer context and RAGAS snippet.
title	text	Human-readable citation title.
uri	keyword	Stable S3 citation.
metadata	object	Governance fields, domain, document type, allowed roles.
checksum	keyword	Source version/change detection.

11. UI Low-Level Design

Screen / State	UI Elements	Behavior
Login	Username, password, Sign in button	Calls `/auth/login`; stores token/session state on success; shows error on failure.
Chat	Message history, chat input, assistant response	Sends `/chat`; displays answer and persists session ID.
Response details	Expander with JSON metadata	Shows sources, tools used, token counts, latency, trace ID, safety, audit event.
Sidebar	New chat, Sign out, previous chats	Clears state or loads `/chat/sessions/{session_id}`.
Error state	Streamlit error/caption	Displays chat/login/history failures without exposing secrets.

12. Logging, Exceptions, And Observability

Concern	Design
CloudWatch logs	ECS containers write application logs to backend/frontend log groups.
Langfuse tracing	Chat requests open an explicit trace/span when Langfuse is configured; fallback trace IDs are local UUIDs.
Callbacks	Langfuse callbacks/config are passed into model calls where supported.
Audit event	Healthcare audit JSON includes redacted query, roles, session, tools, source URIs, trace ID, safety, token usage.
Secrets errors	SecretProvider raises explicit service errors; local/test admin fallback applies only in local/test environments.
Retrieval errors	Search can return empty results; offline/fallback answer tells user when indexed context is missing.
Tool loop safety	Unknown tools return an error string; loop limit forces final answer generation instead of hanging.
Eval publishing	Langfuse score publishing failures are recorded in local report rows without erasing local RAGAS results.

13. Evaluation And Langfuse Score Workflow

RAGAS Evaluation With Langfuse Publishing
1. Eval runner reads a golden dataset and sends each question to `/chat`.
v
2. Each `/chat` response returns answer, source snippets, and real trace ID.
v
3. RAGAS contexts prefer `source.snippet`; URI fallback is used when snippets are absent.
v
4. RAGAS returns faithfulness, answer relevancy, context precision, and context recall.
v
5. Per-question scores publish to the matching Langfuse chat trace.
v
6. A synthetic evaluation-run trace receives average metrics, question count, and failure count.
v
7. Local JSON report records publish status/error fields.

14. Security And Healthcare Controls

Control	Implementation
Authentication	Simple username/password login backed by Secrets Manager password hashes.
Authorization	HealthcareUserContext roles/departments drive source filtering.
Secret handling	Environment variables contain secret names only; secret values remain in AWS Secrets Manager.
PHI protection	PHI patterns are redacted before prompts, traces, logs, and audit payloads.
Source governance	Metadata includes owner, version, effective/review dates, sensitivity, domain, document type, allowed roles.
Clinical safety	Urgent or patient-specific queries are flagged; unsupported clinical advice can be blocked or escalated.
AWS dev admin	Local test admin is disabled by `LOCAL_TEST_ADMIN_ENABLED=false`.
Least privilege	Backend task role reads only required secrets and app data stores; frontend role is minimal.

15. Non-Functional Requirements

NFR	Design Target / Implementation
Security	Private backend, no frontend AWS access, Secrets Manager, IAM least privilege, PHI redaction.
Reliability	Retry wrapper for remote calls; bounded agent loop; local report persists if Langfuse publishing fails.
Availability	ECS Fargate services behind ALB; recommended multi-AZ subnets and rolling deployments.
Performance	OpenSearch top-k retrieval, bounded history context, max five LLM calls, direct S3 manifest for catalog.
Scalability	Stateless services scale horizontally; DynamoDB PAY_PER_REQUEST; OpenSearch Serverless collection.
Maintainability	Modules split by auth, agent, tools, storage, retrieval, ingestion, observability, evals.
Auditability	Trace ID, tools used, sources, safety metadata, token usage, and Langfuse scores retained.
Usability	Single chat UI, session list, response details, and clear fallback messages.

16. AWS Dev Configuration Snapshot

Use the detailed AWS runbook in `docs/aws_setup_instructions.md` for provisioning. The core runtime environment expected by the backend task is shown below.

```
APP_ENV=dev
AWS_REGION=eu-west-2
SECRETS_STAGE=dev
APP_SECRET_NAME=/dstrmaysam-healthcare-knowledge-agent/dev/app
AZURE_OPENAI_SECRET_NAME=/dstrmaysam-healthcare-knowledge-agent/dev/azure-openai
LANGFUSE_SECRET_NAME=/dstrmaysam-healthcare-knowledge-agent/dev/langfuse
S3_BUCKET=dstrmaysam-healthcare-knowledge-agent-dev
S3_RAW_PREFIX=raw/
S3_MANIFEST_KEY=manifests/documents.json
OPENSEARCH_ENDPOINT=https://<collection-id>.eu-west-2.aoss.amazonaws.com
OPENSEARCH_INDEX=dstrmaysam-healthcare-knowledge-agent
DYNAMODB_CHAT_TABLE=dstrmaysam-healthcare-knowledge-agent-dev
CHAT_HISTORY_BACKEND=dynamodb
LOCAL_TEST_ADMIN_ENABLED=false
```

17. Developer Implementation Checklist

1. Provision AWS resources and secrets using the current project names.
2. Create OpenSearch index before running ingestion.
3. Upload approved documents into the S3 `raw/` prefix.
4. Run ingestion with backend environment and task role.
5. Deploy backend and frontend ECS services.
6. Validate login, `/health`, `/documents`, `/chat`, and chat history.
7. Run RAGAS evaluation and confirm per-trace scores in Langfuse.
8. Review CloudWatch logs and Langfuse traces for source coverage and safety metadata.